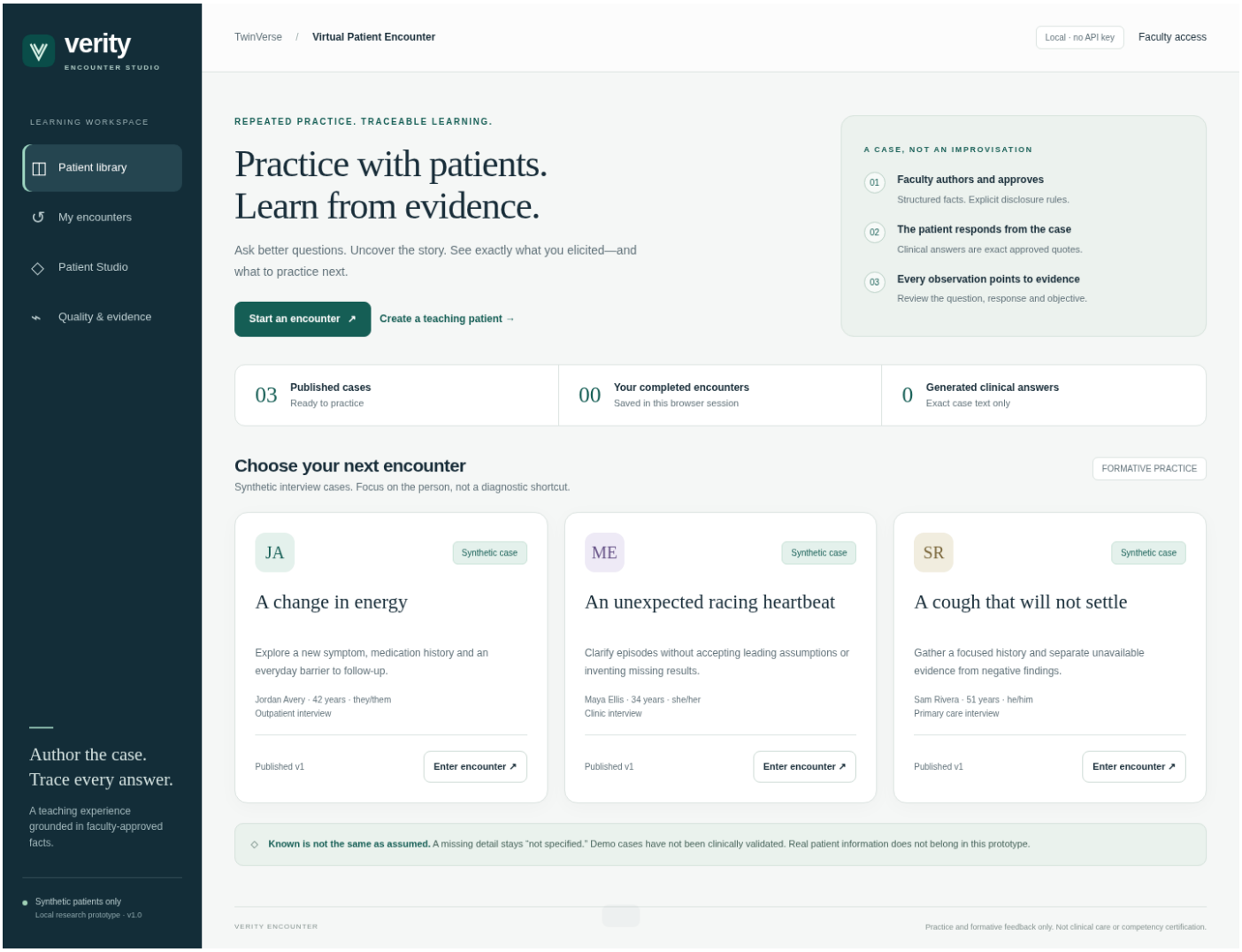
TWINVERSE / CHALLENGE 1

# Verity Encounter

## Author the case. Trace every answer.

A runnable local application for faculty-authored virtual patients, progressive chart disclosure and evidence-linked formative feedback. Built to demonstrate a new, judge-defined patient immediately, without patient-specific code.




Actual application screenshot from the executed Chromium UI test. Three fictional fixtures are included; none is clinician-validated.

## The central design choice

The router selects approved fact IDs; it cannot write a clinical answer. The server enforces disclosure rules and renders the faculty's exact patient or report text. Missing information remains explicitly unspecified. This constrains generated clinical wording, but does not prove routing relevance or source accuracy.

**Executed evidence:** 115 automated tests passed; 34 Chromium UI checks passed using a real-HTTP bridge. External model behavior was mocked, not tested with a live API call.

# One command. No API key.

Default mode needs Python, not a cloud account.

Extract **Verity_Encounter_Source.zip**. Open a terminal inside the extracted folder, then run:

```
cd verity_encounter
python run.py
```

Open **http://127.0.0.1:8080** in your browser. The terminal prints a newly generated faculty passphrase. Select **Faculty access** and enter it to author and approve cases. This is a local demonstration gate, not institutional identity verification.

Python 3.10 or newer with SQLite support is required; this release was executed on Python 3.13.5. Use **python3** on macOS/Linux or **py** on Windows as appropriate. No pip dependencies or JavaScript build are required for the app.

| Task | How |
|---|---|
| Port 8080 is occupied | python run.py --port 8090 |
| Use a fresh demonstration database | python run.py --db data/new-demo.sqlite3 |
| Run automated tests | python -m unittest discover -s tests -v |
| Stop the application | Press Ctrl+C in the terminal. |

## The model connection is optional

Local mode uses faculty-defined question phrases. It is not an LLM and may miss paraphrases or context-dependent questions. The included semantic adapter uses the OpenAI Responses API to select IDs under a strict schema. Enable it explicitly with your own API key and supported model identifier:

```
export OPENAI_API_KEY="your-key"
export OPENAI_MODEL="your-supported-model-id"
python run.py --semantic
```

These commands are for macOS/Linux; PowerShell commands are in README.md. API calls may incur charges. The adapter sends question text, labels and routing phrases, which can contain clinical terminology. It excludes patient answer fields and attachment bytes. Keep all data synthetic. The live connection still needs to be tested on your account.

## Storage and scope

Cases and encounters persist in local SQLite without application-level encryption. Anonymous learner identity does not survive a server restart or session expiry, although saved records remain available to faculty. The server is loopback-only: do not publish it, tunnel it or load real patient information.

# Lead with a new patient.

Prove generality before showing the prepared cases.

| Time | Action and evidence to show |
|---|---|
| 0:00-0:40 | **State the constraint.** "Faculty control what the patient knows. Every clinical response comes from the approved case, and every feedback observation points to encounter evidence." |
| 0:40-2:20 | **Ask the judge for a fictional patient and an unusual detail.** Use New patient to enter the profile, opening fact, distinctive answer and question phrases. Link a learning objective. Save the draft, then approve the version. |
| 2:20-3:15 | **Run it immediately.** Ask a matching question. Show the exact approved wording, source ID, chart update and frozen case version. Then ask an unsupported question and show that no answer is invented. |
| 3:15-4:10 | **Demonstrate controlled disclosure.** In a prepared fictional case, request an attached report. Show that the artifact is unavailable before its linked fact is disclosed and available afterward. |
| 4:10-5:00 | **End and inspect.** Show objective coverage, missed critical elements, quoted evidence and the replay slider. Repeat the encounter against the same case snapshot. Close on the new-case creation, not a diagnosis. |

## An example that makes the proof unmistakable

A faculty judge might supply: "Robin repairs birdhouses every Saturday and stopped three weeks ago." Author that exact fictional detail and a question phrase such as "birdhouse workshop." The point is not the medical content: a fact chosen after the application was built becomes usable through the interface without editing the engine.

## Do not overstate the debrief

Objective coverage means that defined elements were elicited. The communication checklist detects visible text patterns and shows its evidence. Neither is a validated measure of empathy, clinical reasoning or competence. The fixtures are teaching-workflow examples, not approved clinical curriculum.

## A defensible closing

"This prototype turns a faculty-approved case into a repeatable encounter. It separates what the model may select from what the patient may say, then makes the feedback inspectable."

For the full script, demonstration questions and reviewer responses, see docs/JUDGE_DEMO.md. Confirm current competition rules on team composition, AI assistance and code eligibility before submitting.

# What passed. What is unproven.

The release evidence is included with the source.

| Area | Executed result and boundary |
|---|---|
| Engine, service, HTTP and provider contracts | **115 tests passed.** Includes source-only responses, hidden-data restrictions, version immutability, duplicate requests, concurrent updates, authorization, uploads, exported-view redaction and provider fallback. |
| Chromium user interface | **34 checks passed; no JavaScript errors.** Included form-based creation, approval and execution of a previously unseen patient, materials, replay, retry, HTML escaping and mobile layout. |
| UI transport limitation | The environment blocked direct browser-to-localhost navigation. Shipped HTML/CSS/JS was rendered in an offline Chromium document; requests were bridged to the real HTTP server. Native networking, download dialogs and cross-browser behavior remain untested. |
| Startup and syntax | The actual run.py entrypoint started in a subprocess and returned HTTP health and page responses. Python compilation and JavaScript syntax checks passed. |
| Not established | No live model API call, clinician validation, educational-effectiveness evaluation, production security assessment, competition ranking or win probability. |

## Before presenting

Run the app in your own browser and rehearse the exact five-minute path. Have a clinical educator review any case you intend to discuss clinically. Test the semantic adapter with your account before demonstrating it as live; keep local mode ready. Confirm organizer eligibility and disclose AI assistance accurately.

## Before a real institutional pilot

Replace demo access with institutional identity, use production hosting and TLS, add privacy and retention controls, encrypted storage and backups, upload scanning and accessibility review. Separately measure routing relevance with blinded questions and assess formative feedback with faculty and learners. The proposed pilot protocol is in docs/ARCHITECTURE_AND_PILOT.md.

## Where to inspect the evidence

```
evidence/unittest_results.txt
evidence/browser_results.json
evidence/cli_smoke.json
evidence/release_results.json
evidence/source_manifest.json
```

The source manifest hashes shipped code files; the local audit chain links events. Neither is an independent audit or a tamper-proof external record. A source-only answer may still be irrelevant or clinically incorrect if the selected fact or authored source is wrong.